

Beyond Token Time

Persistent Multi-Stream Latent World States for Modality-Agnostic Intelligence

Position Paper and Research Proposal

Nako Sung
NAVER Cloud, Hyperscale AI
August 2026

Abstract. Modern language models maintain rich continuous hidden states, yet their dominant computational interface repeatedly serializes those states into discrete tokens and advances according to a single token clock. Continuous latent reasoning shows that intermediate serialization is not always necessary. Multi-stream language models show that several token streams can coexist. Multimodal architectures show that heterogeneous observations can share latent computation. This paper proposes a broader architectural hypothesis: intelligent computation can be organized around persistent structured state, selective event-triggered updates, and modality-specific readout rather than one global token sequence. A Persistent Multi-Stream Latent World State is coordinated by a Latent Event Bus whose event order preserves causal precedence and, where no causal relation exists, uses an explicitly chosen observer-relative chronology. The proposal is positioned against prior work and paired with compute-matched experiments designed to falsify it.

1 Introduction

Autoregressive language models are built around a simple rhythm. Given tokens $x_{\leq t}$, a transformer computes a continuous hidden representation h_t , maps it to a vocabulary distribution, selects a discrete symbol, and continues:

$$h_t \longrightarrow x_{t+1} \longrightarrow h_{t+1}. \quad (1)$$

This rhythm is natural for language generation. It is less obvious why it should be mandatory for continuing internal computation. Coconut directly challenges that assumption by feeding a hidden state back as a subsequent input embedding rather than decoding every intermediate state into a token [1]. Removing intermediate tokens, however, addresses only one restriction. A second remains: computation is still often conceptualized as one sequence advancing under one global step index.

Real environments are not synchronized this way. A conversational agent may listen, update beliefs, plan, speak, and detect interruption concurrently. A robot may update control at high frequency while planning only when needed. A scene contains many entities whose states evolve independently.

Central question. What if the fundamental unit of intelligent computation were not the next token, but the next relevant event?

The proposed contribution is not latent reasoning, parallel streams, multimodality, or reinforcement learning individually. It is the architectural hypothesis that a general agent can maintain persistent, structured, asynchronous latent processes whose updates are triggered by events rather than token production.

2 Modalities as Lenses

Let s denote an underlying world state. A modality m produces an observation

$$o^{(m)} = g_m(s) + \epsilon^{(m)}, \quad (2)$$

where g_m is a modality-specific measurement or projection and $\epsilon^{(m)}$ captures noise and information loss. An RGB image, speech waveform, text description, depth map, or motor signal is not the world itself. Each is a particular lens on it.

Shared latent spaces are not new. Perceiver IO demonstrated a general latent-processing architecture for heterogeneous structured inputs and outputs [2]. The claim here is narrower and more operational: the shared latent representation should persist across interactions, evolve recurrently, and contain multiple processes with distinct update schedules.

Design principle. A modality is an interface to internal state, not necessarily the format of thought.

2.1 Video generation is multi-stream

Video generation makes the multi-stream requirement concrete. When one model renders two actors in conversation, it must preserve actor identity, per-actor pose and motion, turn-taking, voice-to-character binding, lip synchronization, camera motion, shared scene state, and ambient audio over one evolving timeline. Multi-person conversational video research exposes this binding problem directly: several audio streams must remain attached to the correct visible people through time [8]. Native audio-video generators with synchronized dialogue provide a broader system-level example [9].

This capability is strong behavioral evidence that one

generator can coordinate several concurrently evolving processes. It is not, by itself, proof that the implementation contains explicit or independently addressable latent stream slots; the internal representation may remain entangled. The architectural question is whether making those processes persistent, selective, and schedulable would improve control, efficiency, interruption handling, and consistency.

3 From Token Time to Event Time

A standard autoregressive model implicitly defines a clock in which each increment is tied to a token position. Call this *token time*. A general agent may instead maintain audio, vision, planning, memory, and motor processes with different natural update schedules:

$$t_{\text{token}} \neq t_{\text{compute}} \neq t_{\text{world}}. \quad (3)$$

Computation may advance through events $e_1, e_2, \dots$ that activate only relevant state components. A new acoustic observation may update speaker and semantic state. A sudden obstacle may activate vision, planning, and control. If no verbal response is needed, no language token need be produced.

3.1 Observer-relative event order

An event-driven architecture still needs reproducible order. That order must not be confused with causation. Let $e_i \prec_c e_j$ mean that e_i can causally influence e_j . This is a partial order: some event pairs are unrelated.

A chosen observer O and reference frame $\mathcal{F}_O$ assign each modeled world event a coordinate time $t_{\mathcal{F}_O}(e)$. The event bus can then use the lexicographic key

$$K_O(e) = (t_{\mathcal{F}_O}(e), \tau_O(r_e), \iota(e)), \quad (4)$$

where r_e is the local receipt event, $\tau_O(r_e)$ is the observer’s monotonic local clock at receipt, and $\iota(e)$ is a stable final tie-breaker. The induced order is

$$e_i \prec_O e_j \iff K_O(e_i) <_{\text{lex}} K_O(e_j). \quad (5)$$

The frame and clock are admissible only if causal precedence is preserved:

$$e_i \prec_c e_j \implies e_i \prec_O e_j. \quad (6)$$

For causally unrelated events, $\prec_O$ preserves chronology in the selected frame without claiming that one event caused the other. This is a deterministic total extension of the causal partial order, conceptually related to the separation between causal order and logical clock order in distributed systems [6].

The physics analogy needs one qualification. *Proper time* belongs to a clock along one worldline. It can order the

receipt events r_e observed locally by one agent. It does not by itself assign an occurrence order to remote events at different locations. That broader ordering uses coordinate time in a declared reference frame, whose simultaneity convention is part of the model [7].

Invariant precedence, declared chronology. Causal order is preserved when it exists. Otherwise the system records whose frame defines “earlier” and “later.”

3.2 Causal masking is not a world clock

For a causal decoder, the training interface factorizes a sequence as

$$p_\theta(x_{1:T}) = \prod_{t=1}^T p_\theta(x_t \mid x_{<t}), \quad (7)$$

and applies an attention mask $M_{ij} = -\infty$ for $j > i$. A position therefore cannot read subsequent token positions in that forward pass [10]. This is an externally supplied information-flow constraint over the chosen serialization, not a learned physical causal order among the events represented by those tokens.

The distinction is not that learned weights are order-neutral. Gradients shape them under the causal factorization, positional scheme, and data distribution, so they can encode order-sensitive regularities. The narrower claim is that the mask itself is not stored as a model parameter, and no explicit observer frame or world-event ordering follows from the weights alone.

At inference time, a frame specification $c_{\mathcal{F}}$ can condition a readout without changing the weights, just as natural-language instructions and demonstrations can condition model behavior without gradient updates [11, 12]:

$$y_{\mathcal{F}} = D_\theta(Z; c_{\mathcal{F}}). \quad (8)$$

If Z retains information compatible with several viewpoints, $c_{\mathcal{F}}$ may select a frame-consistent serialization. A useful but deliberately non-mechanistic metaphor is *holographic restoration*: the prompt acts like a boundary condition for reconstructing one view from distributed state. The academic claim is only *frame-conditioned latent readout*. It does not imply that the weights literally implement a hologram, contain a complete frame-independent world, or will preserve invariants without targeted training and evaluation.

Architectural constraint versus learned consequence. Causal masking enforces token-position access order. Training under that constraint shapes the weights. A frame prompt then conditions a readout; whether the underlying state supports faithful cross-frame reconstruction is an empirical question.

4 Persistent Multi-Stream Latent State

At ordered event e_n , let the model maintain

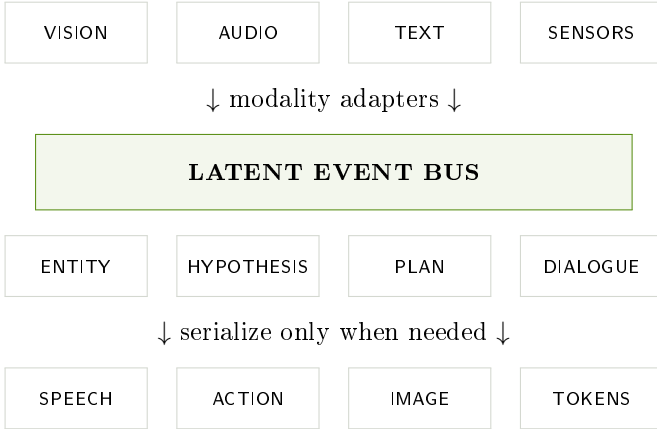
$$Z_n = [z_n^{(1)}, z_n^{(2)}, \dots, z_n^{(K)}]. \quad (9)$$

Streams may correspond to physical entities, goals, memory elements, candidate hypotheses, predicted futures, perceptual processes, or control processes. A routing policy produces an activity mask

$$a_n = \pi_{\text{route}}(Z_n, e_n), \quad a_n \in \{0, 1\}^K, \quad (10)$$

and updates only selected components:

$$z_{n+1}^{(k)} = \begin{cases} F_k(z_n^{(k)}, e_n, Z_n), & a_n^{(k)} = 1, \\ z_n^{(k)}, & a_n^{(k)} = 0. \end{cases} \quad (11)$$



The Latent Event Bus is a common representational and routing layer connecting persistent latent streams, observations, and outputs. Modality-specific encoders and decoders do not define the internal reasoning format.

Latent continuation can be one implementation primitive: an internal action can bypass vocabulary decoding and feed a projected hidden state back into computation. This is closely related to Coconut [1] and is not presented as the primary novelty.

5 Delayed Commitment

If several hypotheses remain plausible, they may be maintained as components of one persistent state rather than instantiated as complete independent textual trajectories. Recent work shows that latent reasoning can also be scaled through parallel stochastic trajectories and latent reward models [4]. The claim is not that continuous representations uniquely enable alternatives. It is that unresolved alternatives may remain coordinated components of a persistent world model.

The defensible advantage is not that a continuous vector contains unlimited information. Discrete output imposes a particular quantization and semantic commitment; delaying that commitment may preserve distinctions useful for later decisions.

6 Relation to Prior Work

Coconut. Continuous hidden-state reasoning and latent alternatives are established [1]. The proposed extension is persistent state across perception, reasoning, and action.

Perceiver IO. General latent processing across heterogeneous inputs and outputs is established [2]. The proposed extension is recurrent persistence with selective event-triggered updates.

Hybrid latent reasoning. Reinforcement learning can optimize latent and discrete reasoning policies [3]. Here it trains update and scheduling policies; RL itself is not the novelty claim.

Parallel latent test-time scaling. Multiple latent trajectories can be sampled and ranked [4]. Here streams may instead be entities, goals, modalities, or hypotheses inside one state.

Multi-Stream LLMs. Several token streams can read and write in parallel [5]. Here internal streams need not be tokens or share one update clock.

Intended novelty. Persistence + structured latent multiplicity + event-driven scheduling + explicit observer-relative event order.

7 Training

Stage I - conventional pretraining. Initialize useful representations and modality adapters through language or multimodal predictive training.

Causal masking at this stage enforces left-to-right access in each serialized training example. It should not be treated as supervision for a unique physical chronology. Frame labels, causal graphs, or multi-frame renderings must be supplied if the system is expected to separate causal invariants from observer-dependent order.

Stage II - latent-state bootstrapping. Introduce recurrence, persistent slots, and routing using next-state prediction, entity tracking, masked-state reconstruction, cross-modal prediction, or distillation.

Stage III - outcome-driven optimization. Reinforcement learning is a natural candidate because many internal trajectories may solve a task. Hybrid latent reasoning establishes RL over latent and discrete policies [3]; here the target policy controls persistent asynchronous state.

A generic objective can reward task success while penal-

izing unnecessary computation, unnecessary serialization, unstable recurrent dynamics, and violations of the declared event order:

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda_c C + \lambda_s S + \lambda_d D + \lambda_o O. \quad (12)$$

8 Experimental Program

The first useful experiment should be deliberately small. Compare four compute-matched systems: (A) sequential discrete reasoning, (B) parallel token streams, (C) sequential latent recurrence, and (D) asynchronous multi-stream latent state.

8.1 Asynchronous multi-entity world

Construct an environment with independently evolving entities and heterogeneous update rates. Measure entity-state accuracy, retention, cross-stream interference, FLOPs, latency, and memory as entity count increases. Shuffle causally unrelated event delivery while retaining frame timestamps; a correct implementation should replay the same observer-relative state trajectory.

8.2 Delayed evidence

Early observations support several hypotheses; later evidence resolves them. Measure hypothesis diversity, contradiction recovery, and revision cost.

8.3 Interruption

Begin an output or action and inject new information before completion. Measure reaction latency, reusable computation, consistency, and unnecessary recomputation.

8.4 Cross-modal action without text

Provide vision and audio and require a non-linguistic action. Compare explicit text-mediated pipelines against direct latent-state-to-action behavior.

8.5 Multi-actor dialogue generation

Generate scenes with two or more actors whose lines overlap, alternate, or are interrupted. Measure identity persistence, speaker-to-voice binding, lip synchronization, turn-order accuracy, cross-actor leakage, and recovery after an unscripted event. Compare an entangled baseline against explicit actor-indexed persistent streams under matched compute.

8.6 Frame and receipt-order ablation

Compare three schedulers: arrival order only, chosen-frame coordinate time only, and the causal-consistent key K_O . Measure replay stability, stale-state writes, causal inversions, and sensitivity to transport jitter. Repeat under alternative declared frames to distinguish invariant causal predictions from frame-dependent chronology.

8.7 Frame-conditioned reconstruction

Serialize the same causal event graph under two or more declared frames, then prompt the model with $c_{\mathcal{F}}$ to reconstruct each view. Hold causal ancestry, entity identity, and outcomes fixed while allowing only the coordinate order of unrelated events to change. Measure invariant accuracy, frame-dependent chronology, and round-trip consistency under $\mathcal{F}_1 \rightarrow \mathcal{F}_2 \rightarrow \mathcal{F}_1$. Ablate the frame prompt, explicit frame supervision, causal mask, and the order of unrelated training events. Failure to preserve invariants would indicate prompt-conditioned narration rather than restoration of a coherent latent event structure.

Killer ablation. If asynchronous structured state does not outperform simpler sequential or parallel baselines under matched compute on tasks designed around independent clocks, interruption, and event-order perturbation, the additional architecture is not justified.

9 Failure Modes and Falsification

- **Stream collapse:** multiple slots converge to effectively identical representations.
- **Fragmentation:** streams become too isolated to maintain global consistency.
- **Latent drift:** recurrence pushes state outside the pretrained distribution.
- **Scheduling overhead:** routing costs more than selective updates save.
- **False concurrency:** parallel slots cosmetically separate one serial computation.
- **Frame leakage:** model predictions that should be causal invariants change when only the declared coordinate frame changes.
- **Prompted confabulation:** a plausible frame-specific narrative is generated without a cross-frame-consistent latent event structure.
- **Order overclaim:** chronological order is misinterpreted as evidence of causation.

10 Discussion

Language models encourage an identification of communication, reasoning, memory, and scheduling with one sequence because text was abundant and next-token prediction was scalable. The architecture through which a capability was discovered need not be the architecture through which it is best organized.

The world does not arrive as a sentence. A room contains many entities simultaneously. Perception need not stop while action occurs. Planning need not stop when new evidence arrives. A hypothesis need not become a sentence before another hypothesis can coexist with it.

Event time should not become another hidden global token counter. The event bus needs a declared perspective: preserve causal precedence, state the observer and reference frame used for chronology, and keep local receipt time distinct from modeled occurrence time.

Question for architecture. Why should language simultaneously serve as interface, reasoning trace, memory representation, scheduler, and computational clock?

11 Conclusion

Continuous latent reasoning shows that models need not verbalize every internal step. Parallel-stream models show that intelligent interaction need not occupy one message stream. Multimodal architectures show that heterogeneous observations can share latent computation. This paper proposes a different primitive: persistent asynchronous state ordered by relevant events.

Compute when an event requires computation. Update the state that the event affects. Preserve causal precedence. Declare the frame used for everything else. Serialize only when interaction requires it.

Tokens are interfaces. Events move the world.

References

- [1] S. Hao, S. Sukhbaatar, D. Su, X. Li, Z. Hu, J. Weston, and Y. Tian, “Training Large Language Models to Reason in a Continuous Latent Space,” arXiv:2412.06769, 2024.
- [2] A. Jaegle, S. Borgeaud, J.-B. Alayrac, et al., “Perceiver IO: A General Architecture for Structured Inputs and Outputs,” ICLR, 2022.
- [3] Z. Yue, B. Jin, H. Zeng, et al., “Hybrid Latent Reasoning via Reinforcement Learning,” NeurIPS, 2025.
- [4] R. You, Y. Li, M. Liu, W. Wang, L. Nie, and W. Li, “Parallel Test-Time Scaling for Latent Reasoning Models,” ACL, 2026.
- [5] G. Su, Y. Yang, X. Li, and J. Geiping, “Multi-Stream LLMs: Unblocking Language Models with Parallel Streams of Thoughts, Inputs and Outputs,” arXiv:2605.12460, 2026.
- [6] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” Communications of the ACM, vol. 21, no. 7, pp. 558-565, 1978.
- [7] A. Einstein, “On the Electrodynamics of Moving Bodies,” Annalen der Physik, vol. 17, pp. 891-921, 1905.
- [8] Z. Kong, F. Gao, Y. Zhang, et al., “Let Them Talk: Audio-Driven Multi-Person Conversational Video Generation,” arXiv:2505.22647, 2025.
- [9] OpenAI, “Sora 2 System Card,” September 2025.
- [10] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” NeurIPS, 2017.
- [11] T. Brown, B. Mann, N. Ryder, et al., “Language Models are Few-Shot Learners,” NeurIPS, 2020.
- [12] S. M. Xie, A. Raghunathan, P. Liang, and T. Ma, “An Explanation of In-Context Learning as Implicit Bayesian Inference,” ICLR, 2022.